

Security Audit

U2U (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	23
Conclusion	24
Our Methodology	25
Disclaimers	27
About Hashlock	28

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The U2U team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

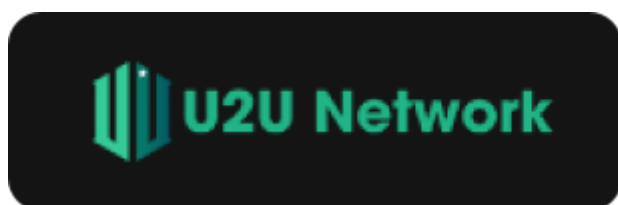
The U2U Network is a decentralized blockchain platform designed to support scalable real-world applications, leveraging a Direct Acyclic Graph (DAG)-based architecture and EVM compatibility for fast, secure, and highly efficient operations. It aims to empower decentralized services, with innovations like DePIN (Decentralized Physical Infrastructure Networks) and modular subnet technology, allowing for high throughput (up to 72,000 TPS) and rapid transaction finality (650ms). U2U focuses on enabling decentralized privacy networks, IoT integration, decentralized storage, and digital identity solutions. This infrastructure aims to drive a decentralized, scalable future for Web3 applications.

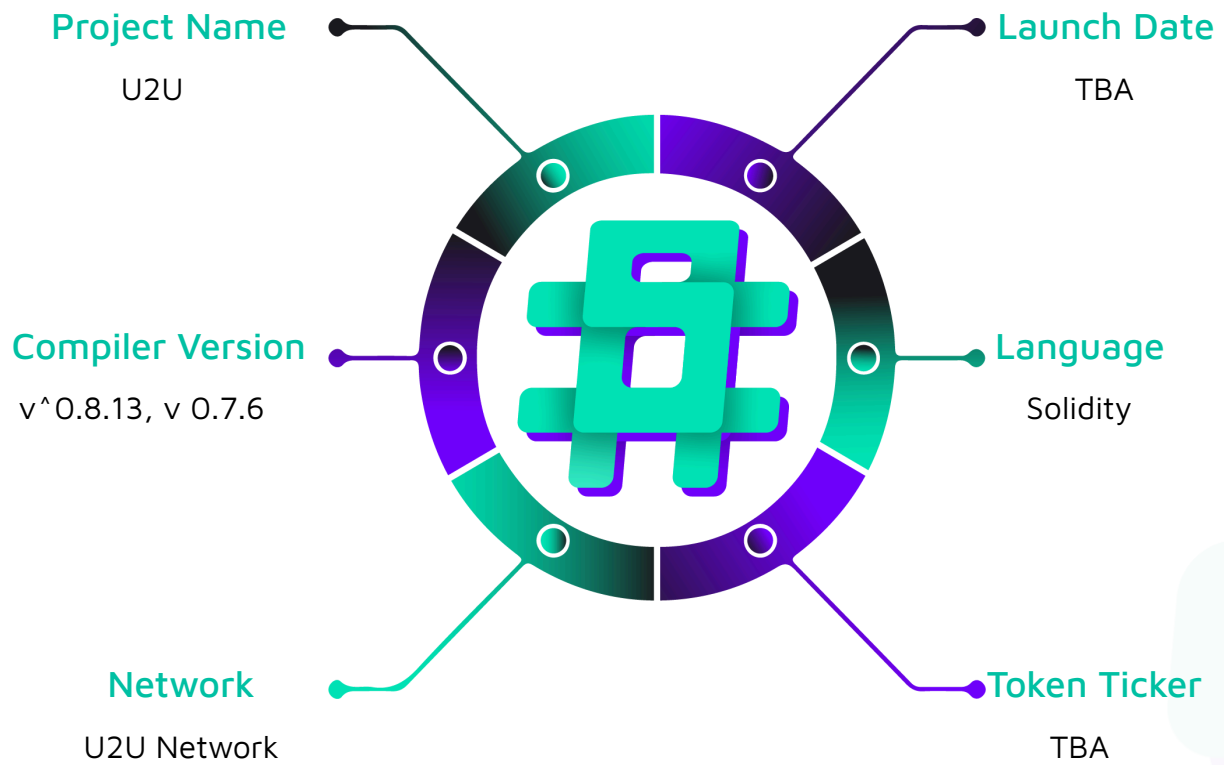
Project Name: U2U

Compiler Version: ^0.8.13, 0.7.6

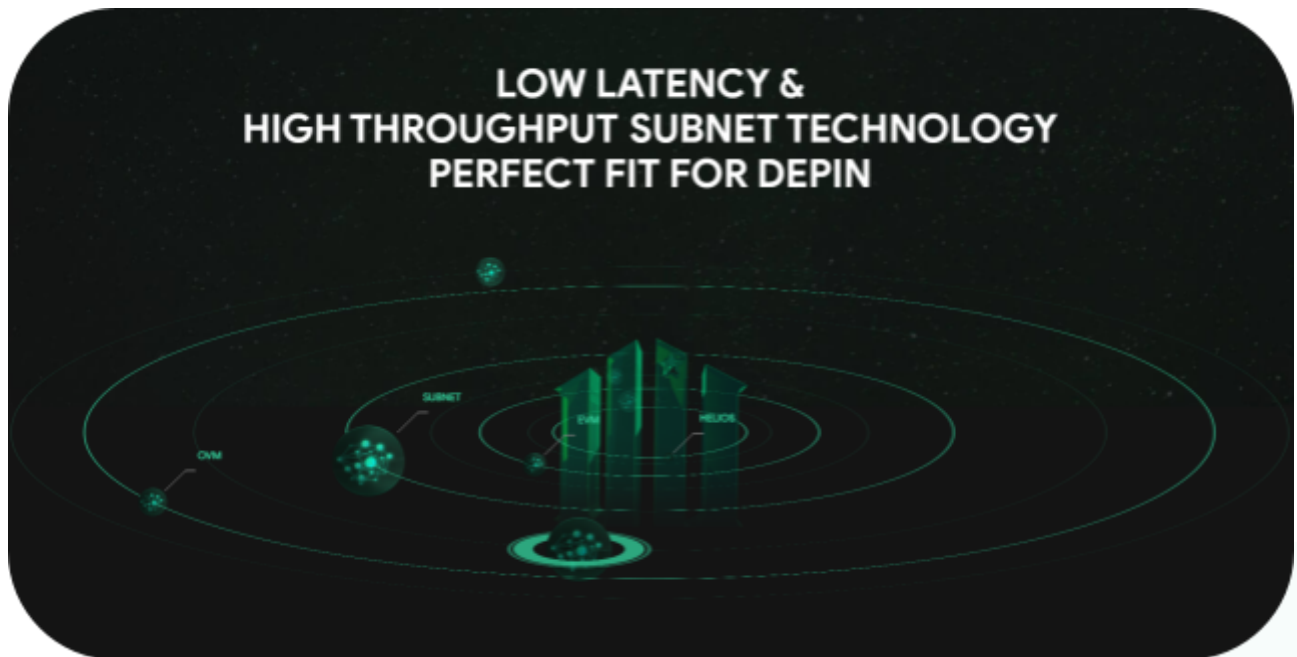
Website: <https://u2u.xyz>

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the U2U project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	U2U Network Smart Contracts
Platform	U2U Network / Solidity
Audit Date	October, 2024
Repository 1	AMM-Contract
Repository 1 Commit Hash	93833866251c697a8c08ac5b0c7a6b647881fa42
Repository 2	AMM-Launchpad-Contract
Repository 2 Commit Hash	cc31946ac518c32cd684fd4364656edc16b91a9d
Repository 3	Market-Contracts
Repository 3 Commit Hash	f6ce0b5096ddca767349f5a2a2932b68aceedc76
Repository 4	Launchpad-Contracts
Repository 4 Commit Hash	0a280f7f48b4587007a66ec27d5c2ae03562041b

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

2 Medium severity vulnerabilities

3 Low severity vulnerabilities

6 QAs

3 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
AMM-Contract DEX smart contracts	Contract achieves this functionality.
AMM-Launchpad-Contract - Allows OPERATOR_ROLE to: Deploy LaunchpadTier, LaunchpadWhitelist, and LaunchpadCommunity contracts - Allows users to: Commit on LaunchpadTier, LaunchpadWhitelist, and LaunchpadCommunity contracts	Contract achieves this functionality.
Market-Contracts Manages NFT listings and allows users to trade NFTs	Contract achieves this functionality.
Launchpad-Contracts Manages NFT listings and allows users to trade NFTs	Contract achieves this functionality.

Code Quality

This Audit scope involves the smart contracts of the U2U project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the U2U project's smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

High

[H-01] launchpad-contracts/U2UBuyBase#claimERC721 - Incorrect value is assigned to the `_amountClaimed[sender]`

Description

The `claimERC721` function updates the `_amountClaimed[sender]` with `ownerOfAmount.length` which is the number of the NFTs to be claimed instead of the accumulated claimed amount.

Vulnerability Details

The `_amountClaimed[sender]` is updated with the same value at every iteration and the incorrect value is assigned.

```
function claimERC721()
    external
    onlyAfterEnd
    onlyAfterStartClaim
{
    ...
    for (uint i = 0; i < ownerOfAmount.length; i++) {
        _amountClaimed[sender] = ownerOfAmount.length;
    }
    ...
}
```

If someone buys/claims repeatedly, `_amountClaimed[sender]` will have the incorrect value after the second claim.

Impact

It makes the `getClaimableAmount` function return the incorrect value.

Recommendation

Increase the `_amountClaimed[sender]` by `ownerOfAmount.length` outside the loop.

Status

Resolved

[H-02] launchpad-contracts/U2UPremintbase#addTokens - Lack of updating `_totalAmountTokens`

Description

The `addTokens` function checks if the sum of `_totalAmountTokens` and `totalNewTokens` exceeds the `_round.maxAmountNFT`, however, the function is missing `_totalAmountTokens` update.

Vulnerability Details

```
function addTokens(LibStructs.Token[] memory tokens)
    public
    onlyOwnerOrRoundContract
    onlyBeforeStart
{
    ...
    require(_totalAmountTokens.add(totalNewTokens) <= _round.maxAmountNFT,
        "U2U: total NFTs can't exceed maxAmountNFT"
    );
    ...
}
```

Since the function doesn't update `_totalAmountTokens`, the variable will always be kept as 0.

Impact

At some point in the future, the total number of tokens added will exceed the maximum amount of the round.

Recommendation

Increase the `_totalAmountTokens` by `totalNewTokens` in the function.

```
function addTokens(LibStructs.Token[] memory tokens)
    public
    onlyOwnerOrRoundContract
    onlyBeforeStart
{
    ...
    require(_totalAmountTokens.add(totalNewTokens) <= _round.maxAmountNFT,
        "U2U: total NFTs can't exceed maxAmountNFT"
    );
    + _totalAmountTokens += totalNewTokens;
    ...
}
```

Status

Resolved

[H-03] market-contracts/NFT#mintNFT, mintBatchNFT, launchpad-contracts/NFT#mintNFT, mintBatchNFT, launchpad-contracts/NFT1155#mintNFT, mintBatchMultipleIdNFT - Lack of access restriction for minting

Description

The above mint functions allow anyone to mint NFTs by missing an access restriction for the functions, allowing anyone mint the NFT at any time.

Recommendation

It's recommended to add access restrictions to the mint functions.

Status

Resolved

Medium

[M-01] market-contracts/ERC721NFTMarketplace#_buy,amm-launchpad-contract/LaunchpadManager#claimTokens,withdrawSoftCap,amm-launchpad-contract/LaunchpadRoundCommunity#cancelCommit,amm-launchpad-contract/LaunchpadRoundTier#claimGiveBack,claimGiveBackFromManager,amm-launchpad-contract/LaunchpadRoundWhiteList#cancelCommit - CEI Pattern Violation

Description

The above functions violate the CEI pattern. External calls are performed before updating the storage.

The CEI pattern helps prevent reentrancy attacks by ensuring that the contract's state is updated before any external interactions take place.

Recommendation

It's recommended to always follow the CEI pattern.

Status

Resolved

[M-02] market-contracts/ERC721NFTMarketplace,ERC721NFTMarketplaceV2,ERC1155NFTMarketplace,RoyaltiesRegistry - Anyone can initialise the implementation contracts

Description

The above contracts are upgradeable smart contracts.

The implementation contracts allow anyone to call the `initialize()` function due to the absence of a `_disableInitializers()` call in the constructor.

Impact

Anyone can take control of the implementation contracts.



Recommendation

It's recommended to add `oz-upgrades-unsafe-allow` constructor.

```
/// @custom:oz-upgrades-unsafe-allow constructor  
  
constructor() {  
    _disableInitializers();  
}
```

Status

Resolved

Low

[L-01] `launchpad-contracts/WETH#withdraw,launchpad-contracts/U2UBuyBase#_transferValueAndFee,amm-launchpad-contract/withdrawSoftCap,transferU2U,withdrawCommit,amm-launchpad-contract/LaunchpadRoundCommunity#commit,cancelCommit,amm-launchpad-contract/LaunchpadRoundTier#cancelCommit, claimGiveBack, claimGiveBackFrommanager` - **Deprecated**
 Ether transfer function

Description

The above functions use the `.transfer()` function to send the native coins.

However, the Istanbul update made some changes to the EVM, which made the `.transfer()` function deprecated for the ETH transfer.

Recommendation

Use `.call()` function to send ether instead of `.transfer()` function and check the return value.

Status

Resolved

[L-02] `market-contracts, launchpad-contracts` - **Outdated version of Solidity**

Description

The project uses `pragma solidity 0.7.6` which is an outdated version of Solidity

Recommendation

Use the latest version of Solidity.

Status

Resolved

[L-03] market-contracts/ERC721NFTMarketplace#setFeeDistributor,market-contracts/RoyaltiesRegistry#initializeRoyaltiesByToken,removeRoyaltiesByToken,launchpad-contracts/U2UBuyBase#setTime, setClaimTime, setPrice, setMaxAmountNFT,setMaxAmountNFTPerWallet,setCollection,setFee,lock,launchpad-contracts/U2UPremintRoundWhitelist#setAdmin,amm-launchpad-contract/LaunchpadManager#updateSnapshotId,amm-launchpad-contract/LaunchpadRoundCommunity#receiptLimitCommit,amm-launchpad-contract/LaunchpadRoundTier#receiptLimitCommit,amm-launchpad-contract/LaunchpadRoundWhitelist#receiptLimitCommit,amm-contract/PancakeV3LmPoolDeployer#deploy - Lack of emitting events

Description

The above functions are missing events when key storages are updated.

Recommendation

Add relevant events based on the variables to be updated.

Status

Resolved

QA

[Q-01] amm-launchpad-contract/LaunchpadManager#claimToken - Lack of a check for the return value on the token transfer

Description

The function is missing a check for the return value on the tokenLaunchpad's transfer.

Recommendation

Add a check if the return value on the token transfer is true.

Status

Resolved

[Q-02] market-contracts/ERC721NFTMarketplace#acceptBid - Incorrect function description

Description

The function name is acceptBid but the description is Create a offer.

Recommendation

Update the description with the correct one.

Status

Resolved

[Q-03] launchpad-contracts/WETH#withdraw - Unnecessary payable keyword

Description

The function is designed to withdraw the native coins but it's set payable.

Recommendation

Remove the payable in the function.

Status

Resolved

[Q-04] launchpad-contracts/NFT,NFT1155,U2UMintRoundFCFS - Unnecessary code blocks**Description**

The contracts have the commented code blocks. Such blocks will reduce the code readability.

Recommendation

Remove the commented code blocks.

Status

Resolved

[Q-05] launchpad-contracts/NFT, NFT1155 - Unused imports**Description**

The NFT contract imports IERC721Metadata but it's not used in the contract.

The same for IERC1155MetadataURI in the NFT1155 contract.

Recommendation

Remove the unused imports.

Status

Resolved

[Q-06] amm-launchpad-contract/LaunchpadManager#depositValue - Lack of implementation**Description**

The depositValue function is declared but it doesn't have the implementation.



Recommendation

Add the function logic.

Status

Resolved

Gas

[G-01] `market-contracts/WETH,amm-launchpad-contract/LaunchpadRoundTier` - Some variables could be made constant

Description

The `name`, `symbol`, and `decimal` variables in the `WETH` contract are only set during the contract deployment.

The same for the `roundType` variable in the `LaunchpadRoundTier` contract.

Recommendation

Make the variables constant.

Status

Resolved

[G-02] `launchpad-contracts/U2UBuyBase#checkOnlyInTimeframe` - `_timeframes.length` is read from storage on each iteration

Description

The `_timeframes.length` is read from storage on each iteration of the loop.

Recommendation

Cache `_timeframes.length` into a memory variable and use the memory variable in the loop.

Status

Resolved

[G-03]amm-launchpad-contract/LaunchpadRoundCommunity#cancelCommit - `userCommit[msg.sender].u2uCommitted` is read from storage multiple times

Description

The `userCommit[msg.sender].u2uCommitted` value is read from storage multiple times even if the function declares the `realCommit` memory variable.

Recommendation

Use the `realCommit` memory variable instead of reading `userCommit[msg.sender].u2uCommitted` value from storage multiple times.

Status

Resolved

Centralisation

The U2U project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlocks analysis, the U2U project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



hashlock.